

Développer en utilisant une BD

Mapper & cie

Géraldine Del Mondo

Département Informatique et Technologie de l'Information

Institut National des Sciences Appliquées - Rouen

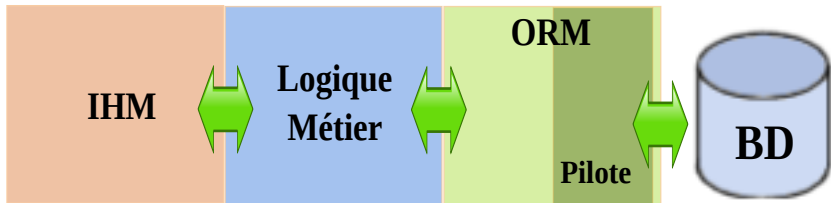
geraldine.del_mondo@insa-rouen.fr

Plan du cours

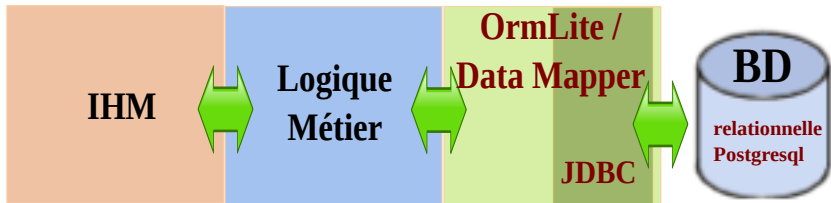
- 1 Positionnement
- 2 Le monde des “Patterns”
- 3 Domain Model & Data Mapper
- 4 Conclusion

OUTILS DE MISE EN ŒUVRE : POSITIONNEMENT

Posgresql, JDBC, ORM



Posgresql, JDBC, ORM



LE MONDE DES “PATTERNS”

Pattern

💡 Qu'est-ce qu'un pattern ?

"Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice" [Christopher Alexander, architect]

- ➔ Un pattern est "half baked"
- ➔ Avoir une connaissance suffisante des patterns existants pour être capable de retrouver le pattern adéquat au problème que l'on se pose dans des livres de référence, où ils sont classés par numéro (catalogues)
- ➔ Les patterns créent un vocabulaire commun qui permet de nommer des catégories de problèmes/solutions et ainsi de mieux communiquer
- ➔ Les patterns ne s'appliquent pas que dans le domaine de l'informatique

Organisation

- Plusieurs catalogues de patterns e.g “Gang of Four” ; “POSA” ; **“P of EAA” : Patterns of Enterprise Application Architecture**

Exemple (résumés) : <https://martinfowler.com/eaCatalog/>

- Eventuellement des liens de dépendance entre les patterns
- Frontière entre les patterns semblent parfois floues

Protocole de modélisation & source de données

“P of EAA”

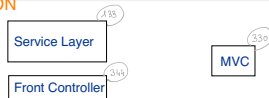
- Construire un modèle du domaine de son application sans penser à la source de données (i.e. pour nous, la source de donnée est la BD)
- Gérer la conception de la base de données juste comme un moyen de persistance des données
- L’ “impedance mismatch”
- Maintenir l’indépendance des couches, un des objectif des patterns : structurer les systèmes
- Adapter la complexité du pattern mis en oeuvre au regard de la complexité du système sur lequel on souhaite l’appliquer ^a
- Le choix du pattern relatif au mapping application-source de données est critique

a. sauf en TP ;-)



Hiérarchie des patterns

PRESENTATION

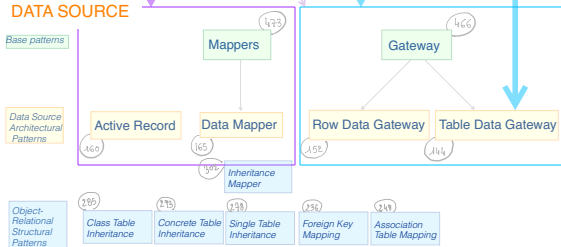


DOMAIN



Domain Logic Patterns

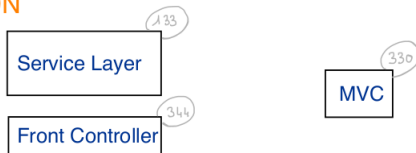
DATA SOURCE



Liste des patterns présentés non exhaustive ; numéros relatifs à [1]

Couche pour organiser la gestion de l'affichage

PRESENTATION



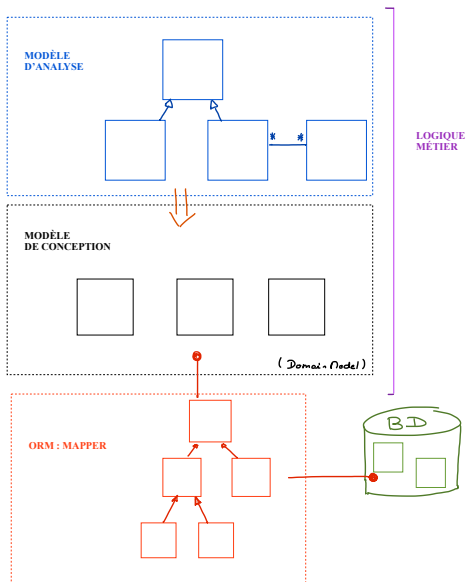
- **Organisation**

- Service Layer [133]
- Front Controller [344]
FC dédié aux applis web

- **Lien avec le Data Model**

- Model View Controller [330]

Point de situation : organiser la logique métier



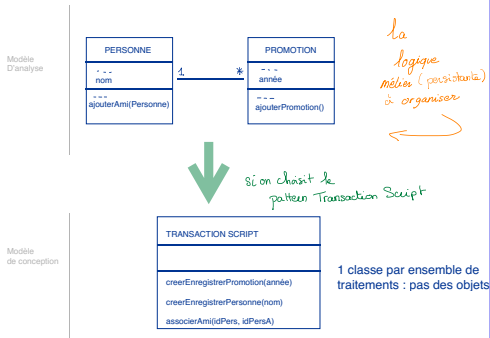
Couche pour organiser la logique métier

DOMAIN



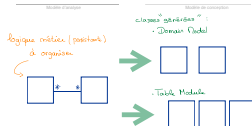
→ choix du DLP

• Transaction Script [110]



CHANGEMENT DE PARADIGME

- Domain Model [116] :
 - pensé indépendamment de la persistance
 - Table Module [125]
 - très proche de la persistance
- (1 table = 1 classe)

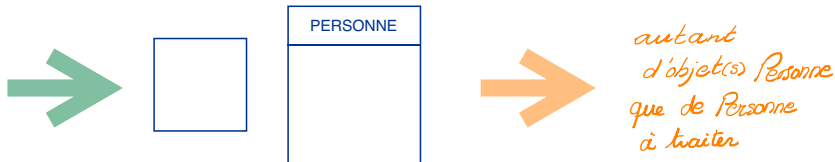


Domain Model versus Table Module

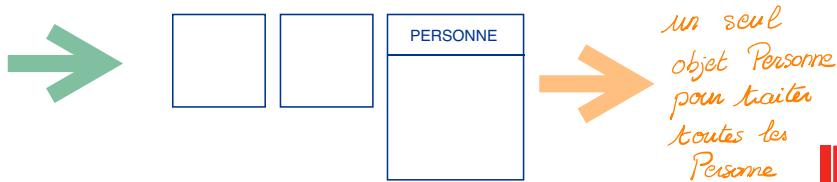
Modèle de conception

classes "générées" :

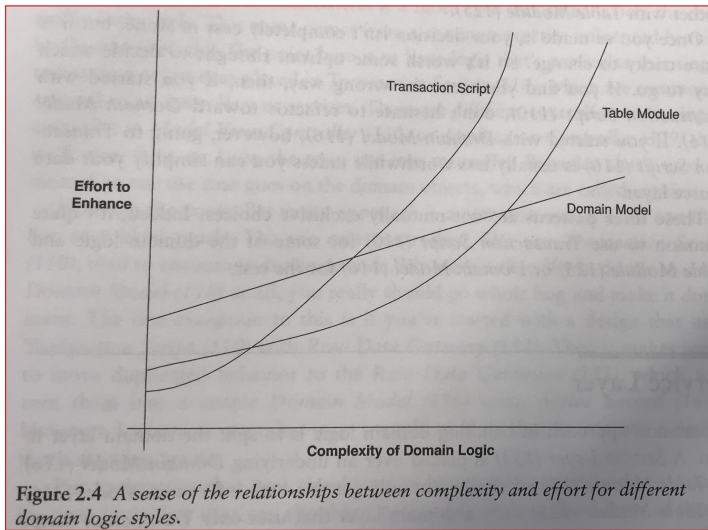
• Domain Model



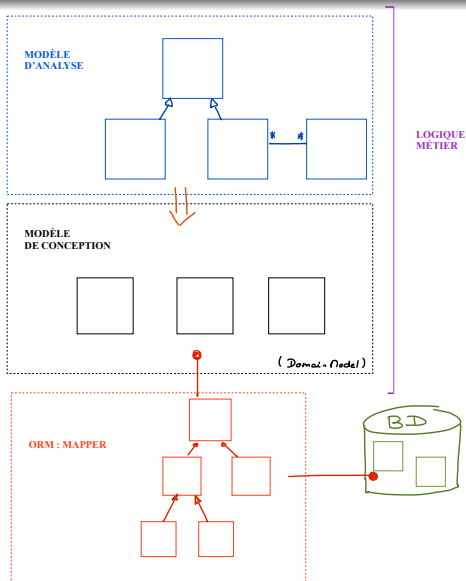
• Table Module : autant de classes que de tables



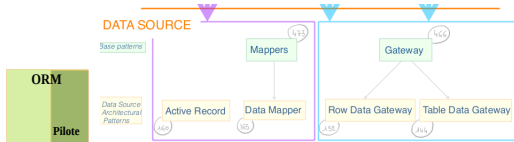
Making a choice [1]



Point de situation : organiser la partie “ORM” - Data Source



Couche pour organiser le lien entre persistance et Domain

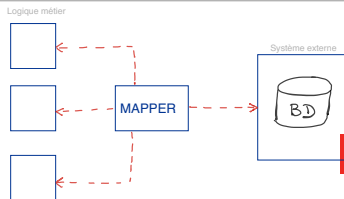
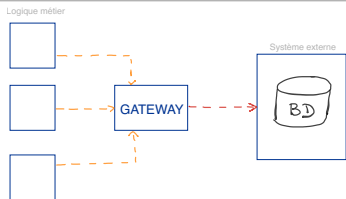


💡 Gateway[466]

Permet de réaliser le mapping entre deux systèmes en encapsulant l'accès à l'un dans l'autre

💡 Mapper[473]

Permet de réaliser le mapping entre deux systèmes qui doivent rester indépendant l'un de l'autre



Gateway - Data Source Architectural Pattern

Toutes les opérations sur l'objet sont du CRUD

Row Data Gateway

152

PERSONNE GATEWAY
nom prénom
insert() delete() update() find()

“Autant d'objets que d'enregistrements”

Table Data Gateway

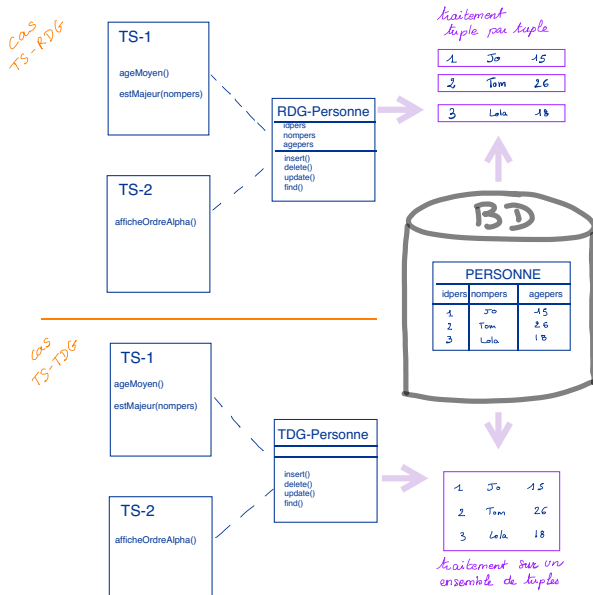
144

PERSONNE GATEWAY
insert() delete() update() find()

“un objet par une table”

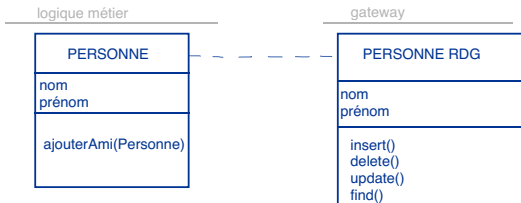
Gateway - Data Source Architectural Pattern : avec TS

Exemple avec un Transaction Script



Gateway - Data Source Architectural Pattern : avec DM

Exemple avec un Domain Model



On ne fera quasiment jamais ce choix :

Si DM simple : Active Record

Si DM complexe : Data Mapper

Pas pertinent avec un Table Module

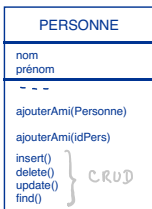
Synthèse des associations entre DM/TM & RDG/TDG

- DM & RDG ✗
- DM & TDG ✗
- TM & RDG ✗
- TM & TDG ✓

AR / Mapper - Data Source Architectural Pattern

Active Record

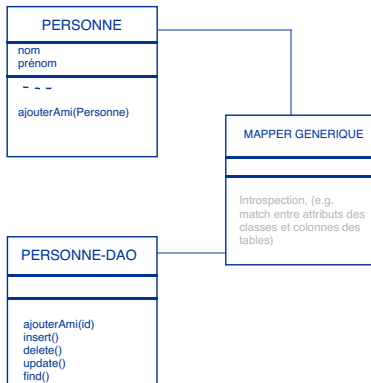
160



Indépendance des couches **×**

Data Mapper

165



Indépendance des couches **✓**

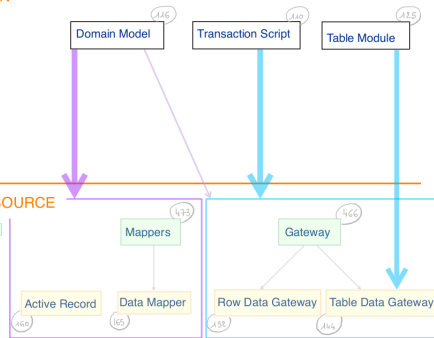
Quelle combinaison Domain & Data Source ?

- Transaction Script & Mapper → ✗
- Transaction Script & Gateway → ✓
- Table Module & Mapper → ✗
- Table Module & Gateway
 - Table Data Gateway → ✓
 - Row Data Gateway → ✗
- Domain Model & Gateway
 - Table Data Gateway → (✗)
 - Row Data Gateway → (✗)
- Domain Model & Active Record →
 - ✓ si DM simple
- Domain Model & Data Mapper →
 - ✓ si DM complexe

DOMAIN

DATA SOURCE

base patterns

Data Source
Architectural
Patterns

DOMAIN MODEL & DATA MAPPER

Domain Model & Data Mapper

Domain Model

Le choix du pattern Domain Model pour la logique métier (ou logique du domaine) fait que le modèle de conception est proche du modèle d'analyse.

Data Mapper OBJECTIF

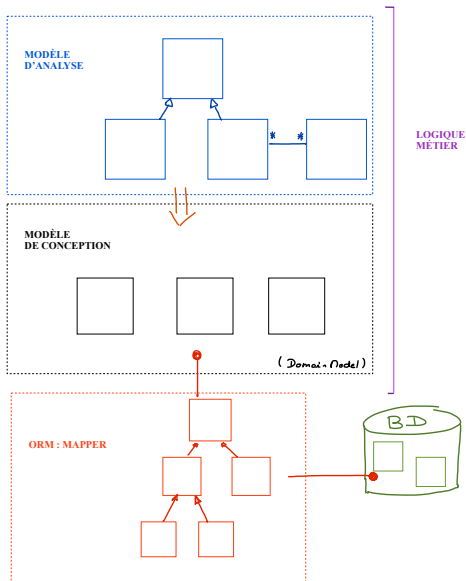
- Cacher la façon dont sont stockées les données à la couche métier
- Garder une indépendance permet d'éviter que des modifications sur la logique métier (resp. la persistance) n'impactent la persistance (resp. la logique métier)
(Souvent implémenté partiellement)

Moyen

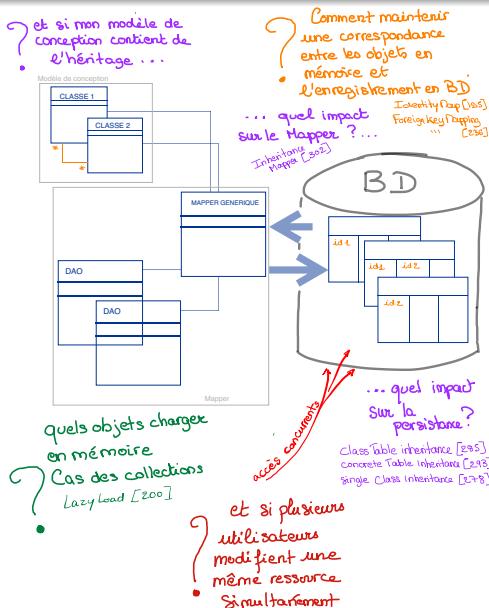
Deux couches : Mapper et métier



Point de situation



Mise en oeuvre du Mapping : quelles problématiques ?



Mise en oeuvre du Mapping : quelles problématiques ?

Problématiques sous-jacentes au mapping

- Le chargement des objets en mémoire :
 - faut -t- il une cohérence totale avec le contenu de la base ?
 - Identity Map [195], Foreign Key Mapping [236], Association Table Mapping [248]
 - faut -t- il tout charger... ou pas ?
 - Lazy Load [200]
 - Et si le modèle de conception contient de l'héritage ?
 - Quel impact sur la persistance de ces objets ?
 - Class Table Inheritance [285]
 - Concrete Table Inheritance [293]
 - Single Class Inheritance [278]
 - Comment les mapper ?
 - Inheritance Mapper [302]
- Quid de la concurrence d'accès aux données ?
 - Optimistic Offline Lock [416]
 - Pessimistic Offline Lock [426]
 - Unit Of Work [184]
 - ...

Identity Map

💡 Identity Map [195]

Qu'est-ce ?

- Une ou plusieurs Map(s)
- Contient les objets relatifs aux données chargées depuis la base

Quelle utilité ?

- Ne pas recharger plusieurs fois un objet
- Gérer les problématiques de concurrence

Comment ?

- Choisir une clé pertinente : e.g. la clé primaire de la table (voir éventuellement Identity Fields [216])
- IM explicite ou générique : une méthode qui cherche tout type d'objet ou une méthode de recherche par type d'objet

Gestion des clés

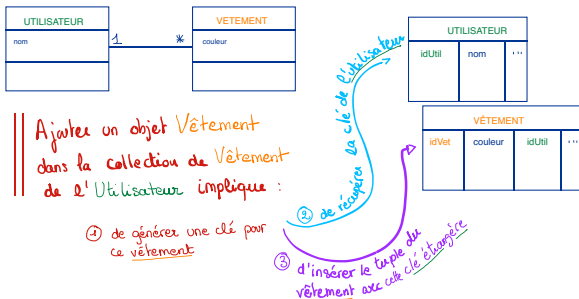
Remarque

Créer un objet c'est générer une clé

Associations

- 1 - * ou 1 - 1 : Foreign Key Mapping [238]
- * - * : Association Table Mapping [248]

➔ Les problèmes de références cycliques : gestion de transactions



Gestion des clés

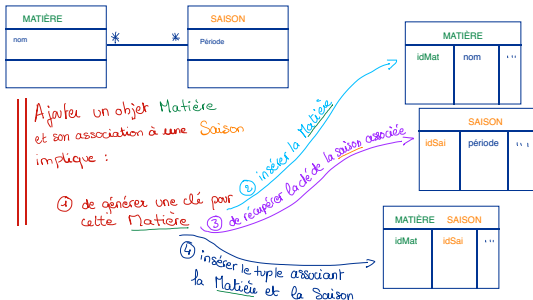
Remarque

Créer un objet c'est générer une clé

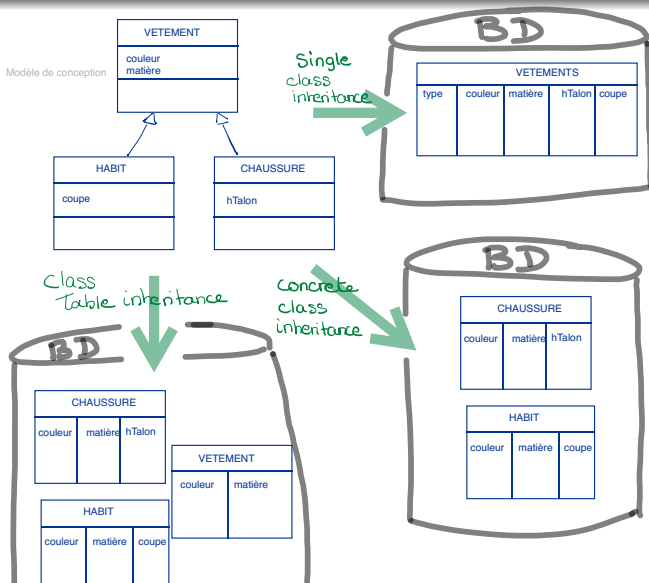
Associations

- 1 - * ou 1 - 1 : Foreign Key Mapping [238]
- * - * : Association Table Mapping [248]

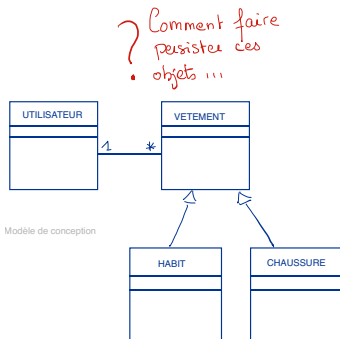
➔ Les problèmes de références cycliques : gestion de transactions



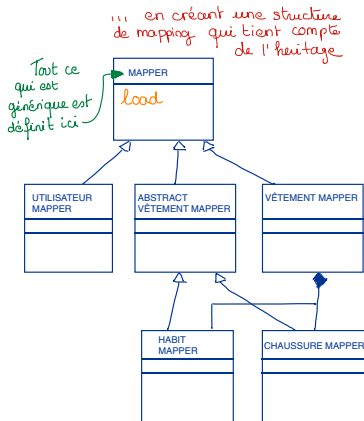
Gestion de l'héritage, gérer l'organisation de la source de données



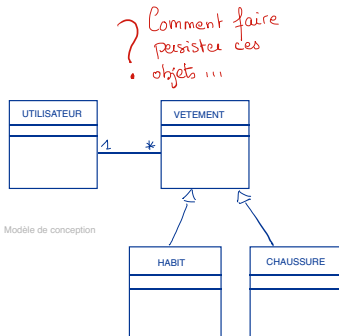
Gestion de l'héritage, gérer le lien avec la logique métier : Inheritance Mapper



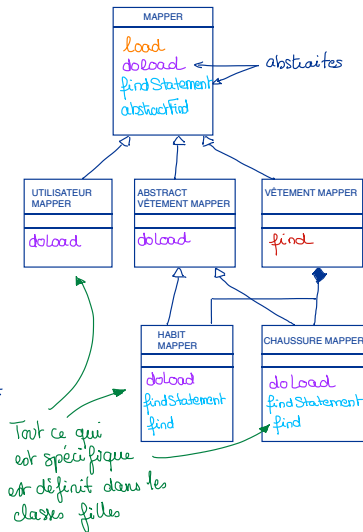
Tout objet que l'on veut charger en mémoire va appeler une méthode load



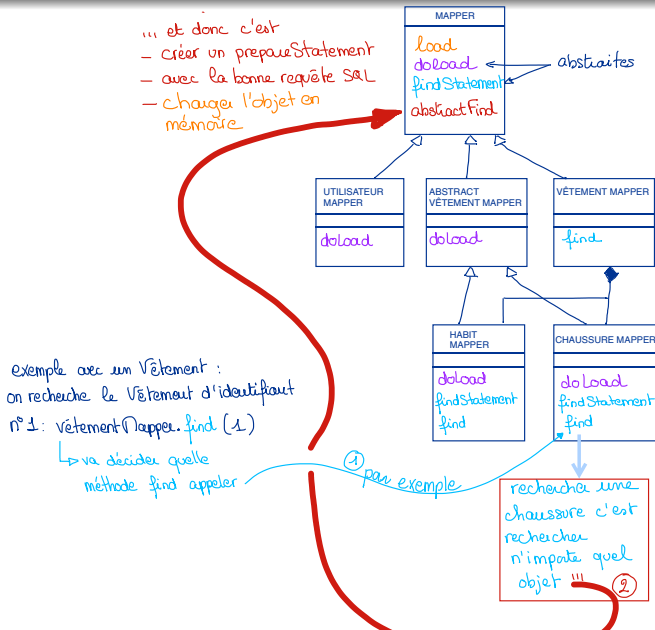
Inheritance Mapper



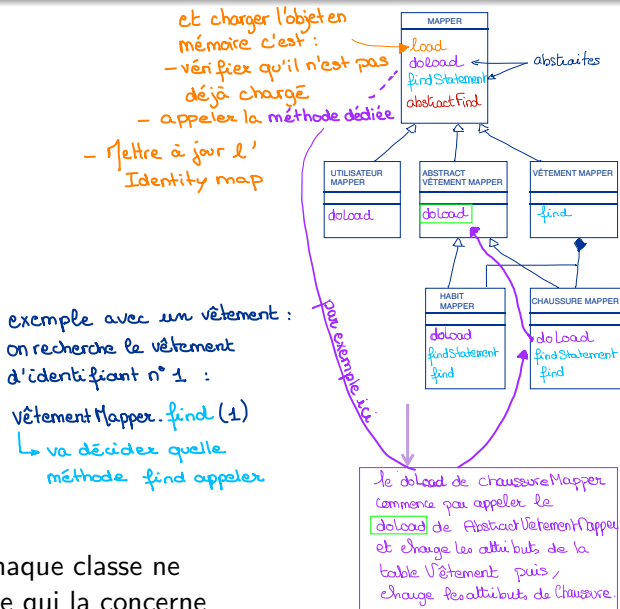
exemple avec un Vêtement :
on recherche le Vêtement d'identifiant
n°1: VêtementMapper.find(1)
↳ va décider quelle
méthode find appeler



Inheritance Mapper



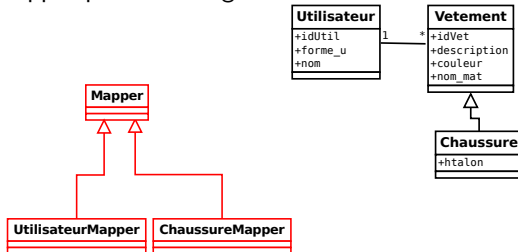
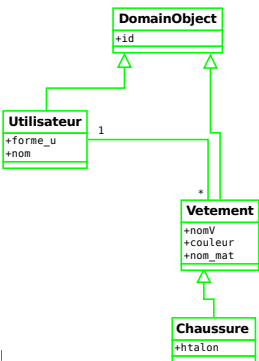
Inheritance Mapper



→ Ainsi chaque classe ne charge que ce qui la concerne

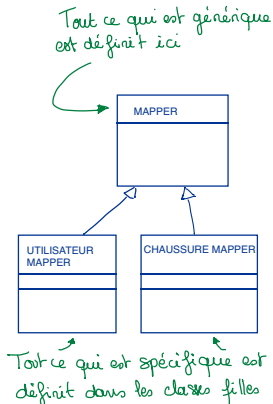
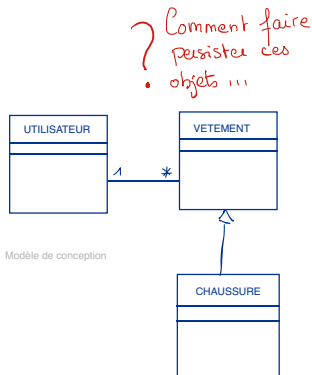
Pour faire le TP, concrètement...

- Nous définirons une Identity Map
- Nous définirons les opérations CRUD pour les classes métier à persister
- Nous mettrons en oeuvre le pattern Foreign Key Mapping pour la gestion des clés étrangères
- Nous définirons un Data Mapper mais ne ferons pas une application stricte du Inheritance Mapper pour l'héritage



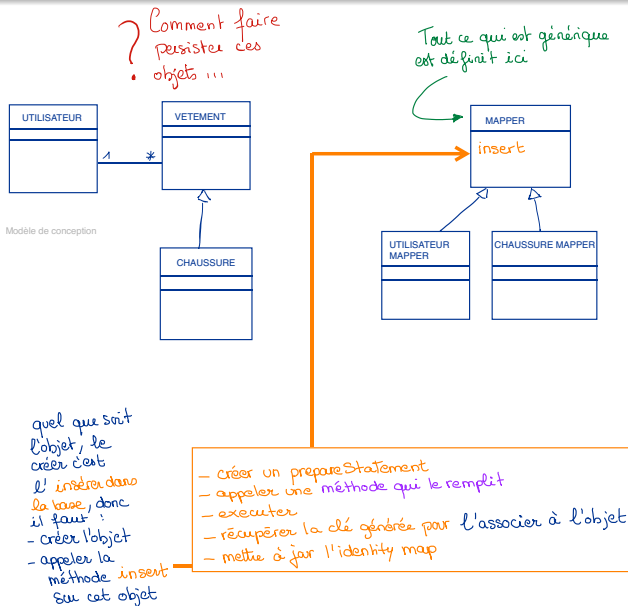
NB : Différence entre la logique métier et le contenu de la base (e.g. utilisateur et chaussure)

Pour faire le TP, concrètement...



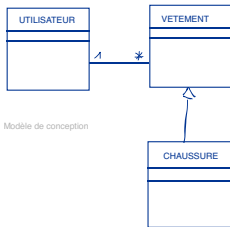
Contrairement au cas général d'application du Inheritance Mapper, ici nous n'allons pas gérer l'héritage en utilisant un Mapper pour Vêtement.

Pour faire le TP, concrètement...



Pour faire le TP, concrètement...

? Comment faire
persistez ces
objets ...

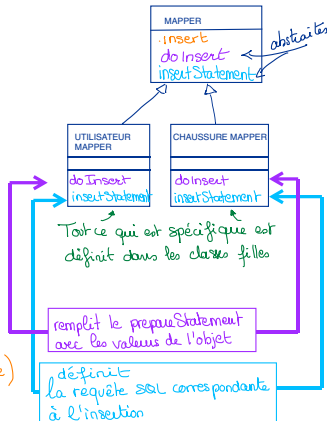


Modèle de conception

Comme les classes filles
héritent de la méthode
insert, on peut par exemple
faire :

chaussureMapper.insert(objetChaussure)

↳ qui de manière
transparente en java
va appeler le doInsert
correspondant avec la
bonne requête SQL en
paramètre



Et les ORMs ?

💡 L'outil ORM

Au delà du concept d'ORM très général vu précédemment, le terme ORM est aussi utilisé pour désigner un outil qui permet de faire le mapping entre un modèle objet et un modèle relationnel de manière "automatique"

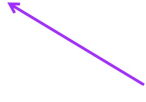
- Un ORM se rattache aux patterns vus précédemment (e.g. Data Mapper, Active Record)
- Il en existe plusieurs, e.g. Hibernate (Data Mapper) ; ORMLite
- Un ORM est quelquefois moins performant que de gérer le mapping "à la main" mais beaucoup plus facile à mettre en oeuvre et à utiliser

Exemple de fichier de Mapping Hibernate : fichier de config

```
<xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-2.0.dtd">

<hibernate-mapping>
  <class name="utilisateur1.Utilisateur" table="utilisateur">
    <id name="idUser" type="long" unsaved-value="null" column="iduser">
<generator class="assigned"/>
    </id>
    <property name="nom" type="string"/>
    <property name="forme_u" type="string"/>
  </class>
</hibernate-mapping>
```

Data Source



Domain



Exemple avec ORMLite : définition d'annotations

```
@DatabaseTable(tableName = "chaussure")
public class Chaussure {

    public static enum Couleur {ROUGE, VERT, JAUNE, FUSHIA, BLEU, MARRON,
        NOIR, BLANC}

    @DatabaseField(generatedIdSequence = "chaussure_idvet_seq")
    private Long idvet;

    @DatabaseField (foreign = true, foreignAutoRefresh = true,
        columnName = "idutil")
    private Utilisateur utilisateur;

    @DatabaseField
    private Couleur couleur;

    @DatabaseField
    private int htalon;

    ...
}
```

Exemple avec ORMLite : utilisation

```
Dao<Utilisateur,Long>utilisateurDao=DaoManager.createDao(connectionSource,  
    Utilisateur.class);
```

```
Utilisateur fb = new Utilisateur();  
fb.setNom("Baucher");  
fb.setForme_u(Utilisateur.Forme.H);  
fb.setChaussures(utilisateurDao.getEmptyForeignCollection("chaussures"));  
  
utilisateurDao.create(fb);
```

```
Dao<Chaussure,Long> chaussureDao = DaoManager.createDao(connectionSource,  
    Chaussure.class);
```

```
Chaussure laChaussDeLaBase = chaussureDao.queryForId(2L);
```

CONCLUSION

Conclusion

Les Patterns du catalogue PEEA

- L'objectif des patterns que nous avons décrits est de proposer des méthodes permettant d'organiser la logique métier d'une application et le mapping (ORM) de cette logique métier avec une source de données persistante.
- Dans ce but soit l'on choisit de lier la logique métier et la persistance, soit on décorrèle les deux. Les décorrélés assure que l'on peut modifier la logique métier ou la persistance sans qu'une modification sur l'un n'ait d'impact sur l'autre.

Les enjeux du mapping

- La complexité du mapping vient du problème de l' "impedance mismatch"
- Il faut faire attention à un certain nombre de problèmes :
 - La cohérence entre les objets et le contenu de la base
 - L'optimisation liée au chargement des objets
 - La gestion de l'héritage
 - La concurrence d'accès aux données

Mapper, l'outil ORM

- Si l'on choisit de partir sur une solution qui maximise l'indépendance entre la logique métier et la persistance alors on s'oriente vers des solutions autour du pattern Mapper
- Gérer ce mapping avec une "solution maison" est complexe, des outils alternatifs existent mais nécessitent une attention particulière pour les optimiser.

Jouons un peu...

SORTEZ VOS TÉLÉPHONES !

wooclap

- Patientez le jeu va se lancer...
 - Il peut y avoir une ou plusieurs bonnes réponses par question.
 - Entre chaque question, vous aurez éventuellement à cliquer sur l'écran pour passer sans attendre à la question suivante.

Bibliographie

[1]Patterns of Enterprise Application Architecture, Martin Fowler, 2003, Addison-Wesley ed.

[2]<https://martinfowler.com/eaCatalog/index.html>,
ddc :05/10/2021